

ICT600 CORRECTED STUDENT SCHEME

February 2026

Question-ordered model answers and marking guide

Scope	February 2026 final examination
Arrangement	Answers follow the original paper from Question 1 through Part B. Coding, diagrams, and written responses are integrated once at their correct positions.
Assessment	100 marks: Part A 70, Part B 30
Status	Corrected student reference; equivalent technically correct answers may receive credit.

How to use this scheme

Question-order rule

Attempt the paper first. Then check the matching answer in this document. Each answer appears once, in the same order as the question paper.

- Equivalent correct wording, diagrams, algorithms, and code may receive full credit when they meet the stated requirements.
- For coding, award method marks when a minor syntax error does not invalidate an otherwise correct solution.
- For design or illustration questions, the labelled diagram is the model answer; prose alone is not treated as a complete design.

Part A

Question 1(a)

8 marks

Describe FOUR applications of HTML.

Mark allocation

Awardable element	Marks
First application	2
Second	2
Third	2
Fourth	2
Total	8

Complete model answer

- Structure web documents.
- Create hyperlinks and embed resources.
- Build forms for input.
- Organise lists, tables, and multimedia content.

Question 1(b)

10 marks

Identify FIVE web-based CMS tools.

Mark allocation

Awardable element	Marks
First CMS	2
Second	2
Third	2
Fourth	2
Fifth	2
Total	10

Complete model answer

- WordPress, Joomla, Drupal, Blogger, and Wix or another valid CMS/site-building platform.

Question 2(a)**6 marks****Describe SIX website-anatomy components.****Mark allocation**

Awardable element	Marks
Containing block	1
Logo	1
Navigation	1
Content	1
Footer	1
Whitespace	1
Total	6

Complete model answer

- Containing block, logo, navigation, content, footer, and whitespace.

Question 2(b)**6 marks****Name SIX client-side development tools or tool categories.****Mark allocation**

Awardable element	Marks
First valid tool	1
Second	1
Third	1
Fourth	1
Fifth	1
Sixth	1
Total	6

Complete model answer

- Any six valid tools or clear category-example pairs, such as JavaScript, jQuery, Bootstrap, React, VS Code, and browser testing tools.

Question 3(a) - create MySQL database

8 marks

Connect, create ict600_db, handle errors and close the connection.

Mark allocation

Awardable element	Marks
Connection	2
Connection error handling	1
CREATE DATABASE statement	1
Query execution	1
Success/query error output	2
Connection closed	1
Total	8

Complete model answer

```
<?php
$conn = mysqli_connect('localhost', 'root', '');
if (!$conn) {
    die('Connection failed: ' . mysqli_connect_error());
}

$sql = 'CREATE DATABASE ict600_db';
if (mysqli_query($conn, $sql)) {
    echo 'Database created successfully';
} else {
    echo 'Database creation failed: ' . mysqli_error($conn);
}
mysqli_close($conn);
?>
```

Question 3(b) - add.php

10 marks

Provide a complete User Entry form and safely insert a new user.

Mark allocation

Awardable element	Marks
Database connection	1
Retrieve and validate inputs	2
Password hash and role/status allowlists	2
Prepared INSERT and execution	3
Success/error output and close	1
Complete form and Back link	1
Total	10

Complete model answer

```
<?php
$message = '';
$conn = mysqli_connect('localhost', 'app_user', 'app_password', 'ict600_db');
if (!$conn) die('Connection failed: ' . mysqli_connect_error());

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $username = trim($_POST['username'] ?? '');
    $password = $_POST['password'] ?? '';
    $role = $_POST['role'] ?? 'user';
    $status = $_POST['status'] ?? 'active';
    $roles = ['user', 'staff'];
    $statuses = ['active', 'inactive'];

    if ($username === '' || $password === '' ||
        !in_array($role, $roles, true) || !in_array($status, $statuses, true)) {
        $message = 'Invalid input.';
    }
}
```

```

    } else {
        $hash = password_hash($password, PASSWORD_DEFAULT);
        $stmt = mysqli_prepare($conn,
            'INSERT INTO users (username, password, role, status) VALUES (?, ?, ?, ?)');
        mysqli_stmt_bind_param($stmt, 'ssss', $username, $hash, $role, $status);
        $message = mysqli_stmt_execute($stmt)
            ? 'New record created successfully' : 'Unable to create record';
        mysqli_stmt_close($stmt);
    }
}
mysqli_close($conn);
?>
<!doctype html><html lang="en"><body>
<h1>User Entry</h1>
<form method="post">
    <label>Username <input name="username" required></label><br>
    <label>Password <input type="password" name="password" required></label><br>
    <label>Role <select name="role"><option>user</option><option>staff</option></select></label><br>
    <label>Status <select name="status"><option>active</option><option>inactive</option></select></label><br>
    <button type="submit">OK</button>
</form>
<?php if ($message != ''): ?>
<p><?= htmlspecialchars($message, ENT_QUOTES, 'UTF-8') ?></p>
<?php endif; ?>
<a href="index.php">Back</a>
</body></html>

```

Marker note

Prepared SQL and password hashing replace the insecure generated answer that concatenated raw input and stored a clear-text password.

Question 4(a) - CodeIgniter Employee controller

4 marks

Construct the CI3 index method that loads employee_profile.php.

Mark allocation

Awardable element	Marks
Employee extends CI_Controller	1
public index method	1
load->view call	1
Correct employee_profile view name/syntax	1
Total	4

Complete model answer

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class Employee extends CI_Controller {
    public function index() {
        $this->load->view('employee_profile');
    }
}
?>

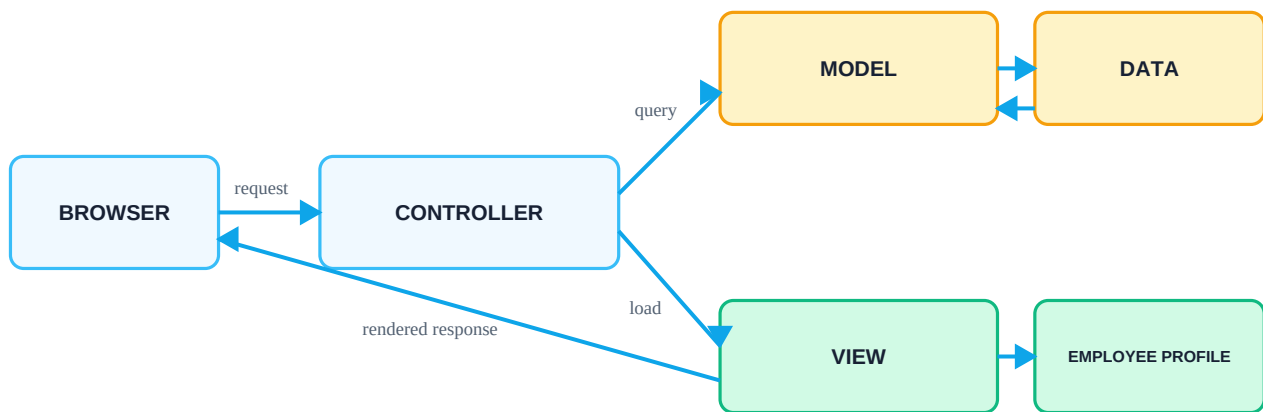
```

Question 4(b)**6 marks****Illustrate the MVC flow from the browser to the Employee controller and Employee Profile view.****Mark allocation**

Awardable element	Marks
Browser request	1
Employee controller and index action	2
Optional model/data step	1
employee_profile view	1
Rendered response	1
Total	6

Complete model answer

- The browser requests the Employee route. The controller index method handles the request, optionally obtains model data, loads employee_profile.php, and returns the rendered Employee Profile response.

**Question 5(a)****6 marks****Assess the clear-text-transmission risk and TWO countermeasures.****Mark allocation**

Awardable element	Marks
Risk assessment	2
HTTPS/TLS	2
Secure configuration/no fallback	2
Total	6

Complete model answer

- Risk: an attacker may intercept or alter credentials and personal data in transit.
- Use HTTPS/TLS with a valid certificate.
- Redirect HTTP to HTTPS and prevent insecure fallback, preferably with HSTS.

Question 5(b)**6 marks****Assess the file-upload risk and TWO countermeasures.****Mark allocation**

Awardable element	Marks
Risk assessment	2
Validation and limits	2
Safe storage and scanning	2
Total	6

Complete model answer

- Risk: a malicious file may execute code, distribute malware, or overwrite/consume server resources.
- Validate actual content/type, extension, and size; rename files.
- Store uploads outside executable web paths and scan them before use.

Part B

Part B Question 1(a) - inputData.html

5 marks

Create the complete bicycle-rental input form.

Mark allocation

Awardable element	Marks
POST form/action	1
Name and phone	1
Three package choices	1
Late-hours numeric input	1
Submit and valid labels	1
Total	5

Complete model answer

```
<!doctype html>
<html lang="en"><body>
<form action="calculate.php" method="post">
  <label>Name <input name="customerName" required></label><br>
  <label>Phone Number <input type="tel" name="phone" required></label><br>
  <fieldset><legend>Package</legend>
    <label><input type="radio" name="package" value="B" required> Basic</label>
    <label><input type="radio" name="package" value="A"> Advanced</label>
    <label><input type="radio" name="package" value="P"> Premium</label>
  </fieldset>
  <label>Late Return <input type="number" name="lateHours" min="0" value="0" required> hour/s</label><br>
  <button type="submit">Submit</button>
</form>
</body></html>
```

Part B Question 1(b) - calculate.php

10 marks

Validate the submitted values, calculate base and late charges, and display the required customer table.

Mark allocation

Awardable element	Marks
Retrieve/validate inputs	2
Package rates	2
Late-charge rule	2
Total calculation	1
Complete required output	2
Safe encoding/valid flow	1
Total	10

Complete model answer

```
<?php
function e(string $v): string { return htmlspecialchars($v, ENT_QUOTES, 'UTF-8'); }
$name = trim($_POST['customerName'] ?? '');
$phone = trim($_POST['phone'] ?? '');
$package = $_POST['package'] ?? '';
$late = filter_input(INPUT_POST, 'lateHours', FILTER_VALIDATE_INT,
    ['options' => ['min_range' => 0]]);
$rates = ['B' => 15, 'A' => 23, 'P' => 100];
if ($name === '' || $phone === '' || !isset($rates[$package]) || $late === false) {
    exit('Invalid input.');
```

```
?>
<table border="1">
<tr><th>CUSTOMER DETAILS</th></tr>
<tr><td><?= e($name) ?></td></tr>
<tr><td><?= e($phone) ?></td></tr>
<tr><td>Package: <?= e($package) ?></td></tr>
<tr><td><?= $late ?> hour/s</td></tr>
<tr><td>Charges: <?= number_format($total, 2) ?></td></tr>
</table>
```

Expected output / behaviour

For Ahmad, package B and 1 late hour: Charges: 25.00

Part B Question 2 - cumulative number analyser

15 marks

Provide the complete HTML and JavaScript page that stores every number and updates the list, maximum and cumulative total.

Mark allocation

Awardable element	Marks
Input, button and three output areas	3
Persistent array	2
Event and numeric validation	2
Append/display all values	2
Maximum	2
Cumulative total	2
Valid complete page and reset/focus behaviour	2
Total	15

Complete model answer

```
<!doctype html>
<html lang="en"><body>
<label>Number: <input id="number" type="number"></label>
<button type="button" id="send">Send</button>
<p>The value you entered: <span id="values"></span></p>
<p>The maximum number you entered: <span id="maximum"></span></p>
<p>The total number you entered: <span id="total"></span></p>
<script>
const numbers = [];
document.getElementById('send').addEventListener('click', () => {
  const input = document.getElementById('number');
  const value = Number(input.value);
  if (input.value.trim() === '' || !Number.isFinite(value)) return;
  numbers.push(value);
  document.getElementById('values').textContent = numbers.join(',');
  document.getElementById('maximum').textContent = Math.max(...numbers);

  document.getElementById('total').textContent =
    numbers.reduce((sum, number) => sum + number, 0);
  input.value = '';
});
</script>
</body></html>
```